

Ontwikkeling van een live race-engineeringdashboard

Jeff Mathieu

KU Leuven
Faculty of Engineering Science
Master of Engineering: Computer Science
Option: Human-Machine Communication

Administratieve gegevens

Opleidingsonderdeel:	Industriële Stage: Computerwetenschappen (<i>H04I9A</i>)
Stageplaats:	MotorSport Training Center (MSTC)
Adres:	Rerum Novarumlei 19 - 2930 Brasschaat
Dagelijkse begeleider:	Jan Wouters info@motorsport-trainingcenter.be +32 475 39 41 16
Stageperiode:	22 juni - 31 juli 2026
Academiejaar:	2026-2027

30 augustus 2026

Inhoudsopgave

1	Inleiding en context	1
1.1	Motorsport en Endurancewedstrijden	1
1.2	Stageplaats en opdracht	1
1.3	Positie van MSTC binnen de Belgische motorsport	2
1.4	Doelstellingen	2
2	Van stageplan naar iteratieve ontwikkeling	3
2.1	Oorspronkelijke planning en requirements	3
2.2	Werkelijke iteratieve werkwijze	4
2.3	Begeleiding en feedbackmomenten	4
2.4	Vergelijking met het stageplan	4
3	Technische architectuur, alternatieven en lokale opslag	5
3.1	Technische architectuur	5
3.1.1	Electron	5
3.1.2	Proces- en modulegrenzen	5
3.1.3	Vensterarchitectuur	6
3.1.4	Datastroom	6
3.1.5	Instellingen en configuratie	7
3.2	Afweging van alternatieve oplossingen	7
3.2.1	Webapplicatie	7
3.2.2	SQLite	7
3.2.3	Herberekende statistiek	7
3.2.4	Machine learning model	8
3.3	Dataverwerking en lokale opslag	8
3.3.1	Uniforme parser	8
3.3.2	Vlagstatus en pitfasen	8
3.3.3	Bestandsformaten	8
4	Analyse, voorspelling en strategie	9
4.1	Ronde- en pilootstatistieken	9
4.2	Voorspelling van de actuele ronde	10
4.3	Referentietijden	10

4.4	Vergelijkingen binnen de klasse	10
4.5	Pitstopplanner	11
4.6	Full Course Yellow	11
4.7	Automatische rapportage	11
5	Praktijktesten op het circuit	12
5.1	Context van het eerste raceweekend	12
5.2	Uitgevoerde tests en verzamelde data	12
5.3	Belangrijkste bevindingen	13
5.3.1	Vlagcontext en geldige rondes	13
5.3.2	Gemiddelden en vergelijkingseenheden	13
5.3.3	Gaps en pitstopprojectie	13
5.4	Nieuwe requirements uit de praktijktest	14
5.5	Prioriteiten na Spa	14
5.6	Aanvullende tests tussen Spa en Zolder	14
5.7	Het einddoel: <i>24 Hours of Zolder</i>	15
5.7.1	Verwachte meerwaarde tijdens de race	15
5.7.2	Wat tijdens de 24 uur getest moest worden	15
5.7.3	<i>24 Hours of Zolder</i> : het verloop van de race	16
6	Evaluatie en persoonlijke reflectie	17
6.1	Evaluatie van het stageplan	17
6.2	Zelfstandig werken	17
6.3	Communicatie en requirementsanalyse	17
6.4	Software-engineeringvaardigheden	17
6.5	Onverwachte competenties	18
6.6	Wat beter kon	18
7	Relatie met de opleiding	19
7.1	Objectgericht programmeren: <i>H01P1A</i>	19
7.2	Software-ontwerp: <i>G0Q40C</i>	19
7.3	Software Architecture: <i>H07Z9B</i>	20
7.4	Fundamentals of Human-Machine Interaction: <i>H0V14A</i>	20
7.5	Bedrijfservaring: Computerwetenschappen: <i>H04G0A</i>	21
8	Conclusie	21

A	Aanvullende afbeeldingen	23
B	Voorbeeld van automatisch gegenereerd raceverslag	25
	Afkortingenlijst	26
C	Stageplan	27
D	Competenties die ik verwachtte te verbeteren	28

Hoofdstuk 1

Inleiding en context

1.1 Motorsport en Endurancewedstrijden

Motorsport is een van de meest complexe en dynamische wedstrijdgevingen. Een endurancewedstrijd maakt dit nog complexer door de lange duur, meerdere piloten per wagen, verplichte pitstops binnen bepaalde tijdsvensters, neutralisaties van de race, wisselde weersomstandigheden, verschillende klassen van wagens die tegelijk op de baan zijn, de gevreesde referentietijd¹ en de noodzaak om tijdens de race een voorbepaalde strategie te volgen en aan te moeten passen naarmate de race vordert. Dit laatste aspect maakt endurancewedstrijden, ook voor mij persoonlijk, zeer interessant aangezien beslissingen in real-time tijdens de race genomen moeten worden.

Het Belcar Endurance Championship is een Belgische endurancecompetitie waarin teams gedurende langere races niet alleen op snelheid, maar ook op strategie, betrouwbaarheid en consistente rondetijden worden beoordeeld. Omdat verschillende klassen tegelijk op hetzelfde circuit rijden, is het voor een team belangrijk om vooral de relevante tegenstanders binnen de eigen klasse correct op te volgen. Voor MSTC, dat met wagen 33 uitkomt in de Club Challenge-klasse, betekent dit dat live informatie over positie in klasse, rondetijden, sectortijden, pitstops en verschillen met klassegenoten cruciaal is om tijdens de race onderbouwde beslissingen te nemen.

Wat het nu dus moeilijk maakt is hoe de timinginformatie van de officiële timingpaginas van de endurancewedstrijden een globaal overzicht geven van de race en dus niet enkel van de klasse die we volgen. Zoals we in voorbeeld A.1 kunnen zien, zitten alle klassen door elkaar en is het dus moeilijk om te volgen waar de directe concurrenten zich bevinden. De timingpagina is dan ook ontworpen voor een algemeen publiek en voor het wedstrijdverloop als geheel. Maar voor het team is het belangrijk om te vergelijken hoe de huidige piloot het doet ten opzichte van zijn teamgenoten, hoe het tempo van de andere auto's in de klasse evolueert, waar de wagen vermoedelijk zal uitkomen na een pitstop en of een voorspelde rondetijd de opgelegde referentietijd nadert. Om deze informatie allemaal handmatig uit deze algemene timingpagina te halen is bijna onmogelijk.

1.2 Stageplaats en opdracht

Jan Wouters, mijn dagelijkse begeleider tijdens deze stage, is de oprichter van MotorSport Training Center (MSTC). MSTC is een bedrijf dat zich richt op het trainen van piloten en teams in de motorsport. Naast het aanbieden van trainingen en coaching, ondersteunt MSTC ook teams tijdens (endurance)wedstrijden.

Jan zelf is al jarenlang een vaste waarde binnen de Belcar-omgeving en het Belgische racecircuit. Tijdens raceweekends neemt hij een brede rol op: hij bereidt de organisatie mee voor, stemt af met stewards en race control, coacht piloten tijdens de wedstrijd en neemt strategische

¹Binnen het *Belcar Endurance Championship* geldt per klasse een minimale referentierondetijd die niet onderschreden mag worden. Wanneer een wagen sneller rijdt dan deze limiet, kan dit leiden tot een tijdstraf of, in ernstigere gevallen, tot diskwalificatie. Daarnaast kan de wagen verplicht worden om in een hogere klasse uit te komen, wat competitief nadelig is omdat er dan tegen snellere wagens gereden moet worden.

beslissingen op basis van de beschikbare timinginformatie. Die ervaring en reputatie maken hem tot een belangrijke figuur binnen de Belgische endurancerwereld. Zelf kijk ik al lang op naar wat Jan binnen de Belgische motorsport heeft opgebouwd, onder meer met meerdere klasseoverwinningen in de *24 Hours of Zolder* en sterke resultaten in het *Belcar Endurance Championship*. Het was daarom een grote eer om mijn stage bij hem te mogen doen en met deze opdracht een concrete bijdrage te leveren aan de werking van MSTC en het team.

Door zijn ervaring werd Jan ook benaderd door Bert Longin, eveneens een bekende naam in de Belgische motorsport, om mee een team van bekende Vlamingen te begeleiden in de voorbereiding op en tijdens de *24 Hours of Zolder 2026*. Voor die race zal Jan niet alleen wagen 33 van MSTC opvolgen, maar dus ook deze tweede wagen coachen. Het gelijktijdig opvolgen van meerdere wagens tijdens een endurancewedstrijd is bijzonder moeilijk wanneer men enkel beschikt over de standaard timingpagina. De timingpagina bevat wel de officiële tijden en posities, maar biedt weinig ondersteuning voor teamgerichte vergelijkingen, stintopvolging, pitstopplanning en strategische interpretatie. Vanuit die praktische nood ontstond de vraag om een dashboard te ontwikkelen dat deze live timingdata omzet in bruikbare informatie voor Jan zelf.

Het concrete doel van de opdracht was dus om een desktopdashboard te ontwikkelen dat tijdens raceweekends naast de officiële timingpagina kan worden gebruikt. De applicatie moest live timingdata opvolgen, relevante racehistoriek lokaal bewaren en deze informatie vertalen naar inzichten die specifiek nuttig zijn voor het eigen team. Daarbij ging het niet enkel om het tonen van tijden, maar vooral om het ondersteunen van beslissingen rond tempo, stintopvolging, klassevergelijkingen en pitstopstrategie. Omdat het dashboard tijdens wedstrijden voornamelijk op Windows gebruikt zal worden, maar tijdens de stage hoofdzakelijk op macOS werd ontwikkeld, waren platformafhankelijkheid, eenvoudige installatie en betrouwbaar herstel na een onderbreking vanaf het begin belangrijke aandachtspunten.

1.3 Positie van MSTC binnen de Belgische motorsport

MSTC combineert pilotenbegeleiding en training met ondersteuning tijdens raceweekends. De mogelijke klanten bestaan voornamelijk uit amateur- en semiprofessionele piloten en kleinere raceteams die nood hebben aan coaching en strategische ondersteuning.

Mogelijke concurrenten zijn andere raceopleidingen, zelfstandige pilootcoaches en teams die zelf vergelijkbare begeleiding aanbieden. MSTC onderscheidt zich vooral door de combinatie van training en directe ondersteuning tijdens wedstrijden. De domeinkennis van Jan Wouters en zijn ervaring binnen het *Belcar Endurance Championship* zorgen ervoor dat de begeleiding nauw aansluit bij concrete situaties op het circuit.

De regionale context speelt daarbij een belangrijke rol. MSTC is sterk verbonden met Circuit Zolder en de Belgische endurancerwereld, met de *24 Hours of Zolder* als het belangrijkste evenement. Ook wedstrijden op circuits zoals Spa-Francorchamps maken deel uit van deze omgeving. Deze lokale aanwezigheid zorgt voor korte communicatielijnen en gespecialiseerde ondersteuning, maar betekent tegelijk dat veel kennis bij een beperkt aantal personen zit.

1.4 Doelstellingen

De uiteindelijke opdracht bestond uit de volgende hoofddoelen:

- Live timingdata van meerdere websites² betrouwbaar lezen en naar één intern formaat vertalen.
- Een historiek van rondes, sectortijden en sessiecontext lokaal opslaan.
- Meerdere wagens kunnen volgen zonder de timingwebsite meerdere keren te bewaren.
- Analyses berekenen voor race, kwalificatie en vrije training.
- Actuele rondevoorspellingen en configureerbare referentietijden tonen.
- Een pitstopvenster en een projectie van de positie na een pitstop aanbieden.
- Neutralisaties, pitstops, outlaps en onvolledige data correct behandelen.
- De logica modulair en automatisch testbaar maken.
- Installaties en updates voor macOS en Windows via GitHub Releases ondersteunen.

Hoofdstuk 2

Van stageplan naar iteratieve ontwikkeling

Het oorspronkelijke stageplan beschreef een grotendeels opeenvolgend stappen van analyse, dataverwerking, dashboardontwikkeling, simulatie, voorspelling en testen. In de praktijk werd deze weekplanning slechts beperkt gevolgd. De algemene doelstelling bleef wel behouden: een lokaal dashboard ontwikkelen dat live timingdata omzet in bruikbare informatie voor een race-engineer. De concrete requirements, technische keuzes en prioriteiten veranderden wel tijdens de stage.

2.1 Oorspronkelijke planning en requirements

De eerste week was voorzien voor requirementsanalyse en onderzoek naar de beschikbare timingdata. Daarna zouden lokale opslag en replay worden ontwikkeld, gevolgd door het dashboard en de basisanalyses. De vierde week legde de nadruk op robuustheid en simulatie. Voor de vijfde week was een AI- of ML-gebaseerd voorspellingsmodel gepland en de laatste week was bestemd voor testing, documentatie en oplevering.

De initiële requirements volgden dezelfde indeling. De app moest timingdata verzamelen en lokaal bewaren, historische of gesimuleerde sessies kunnen afspelen, rondetijden en sectoren analyseren, voorspellingen maken en de resultaten overzichtelijk tonen. Voor de opslag werd aan SQLite gedacht en voor de voorspelling aan een rolling average of regressiemodel. Deze requirements beschreven vooral het gewenste eindresultaat. De precieze schermindeling, de betekenis van verschillende timingvelden en de regels voor geldige analyses waren bij de start nog niet volledig bepaald. Het stageplan kon daardoor als vertrekpunt dienen, maar niet als volledige technische specificatie.

²Aangezien de races van de Belcar Endurance Championship op meerdere circuits gereden worden, zijn er circuitafhankelijke timingpaginas die gebruikt worden. Het is dus belangrijk dat het niet uitmaakt welke timingpagina gebruikt wordt voor het dashboard zelf.

2.2 Werkelijke iteratieve werkwijze

De ontwikkeling zelf verliep als een herhaalde cyclus van overleg, implementatie, testen en bijsturen. Een concrete vraag van de opdrachtgever werd eerst vertaald naar een prototype of een kleine technische oplossing. Vervolgens werd gecontroleerd of de oplossingen werkten met synthetische data, historische racedata en/of een live timingdemo. De bevindingen uit die controle bepaalden welke aanpassing daarna de hoogste prioriteit kreeg.

De requirements bleven daardoor gedurende de volledige stage evolueren. Sommige wensen konden pas worden beoordeeld wanneer ze zichtbaar waren op het dashboard. Andere vereisten veranderden omdat realistische timingdata meer uitzonderingen bevatte dan de eerste testscenario's. Een afwijking leidde daarom niet alleen tot een correctie in de code, maar soms ook tot het opnieuw formuleren wat de applicatie precies moest berekenen of tonen.

Deze werkwijze betekende dat de geplande weekfasen sterk door elkaar liepen. Analyse, implementatie en testing gebeurden vaak binnen dezelfde iteratie. Vooral na de praktijktest in Spa-Francorchamps kregen betrouwbaarheid en correctheid voorrang op uitbreidingen die oorspronkelijk voor die bepaalde week waren voorzien.

2.3 Begeleiding en feedbackmomenten

Omdat de begeleider vaak op verplaatsing werkte, gebeurde het grootste deel van de technische ontwikkeling zelfstandig van thuis uit. De begeleiding bestond daarom niet uit dagelijks fysiek overleg. De voortgang werd regelmatig besproken via e-mail, telefoon, met fysieke overlegmomenten en feedback tijdens raceweekends. Tijdens deze contactmomenten leverde Jan vooral domeinkennis vanuit zijn ervaring als race-engineer en coach. Hij verduidelijkte onder meer welke informatie tijdens een race belangrijk was, hoe bepaalde reglementen geïnterpreteerd moesten worden en welke weergave praktisch bruikbaar was. De technische implementatie en feedback werd vervolgens zelfstandig uitgewerkt.

Werkende versies, schermafbeeldingen en concrete racescenario's bleken het meest geschikt om feedback te verzamelen. Een zichtbaar dashboard maakte sneller duidelijk welke informatie ontbrak of verwarrend werd weergegeven dan een, al dan niet verwarrende, technische beschrijving.

2.4 Vergelijking met het stageplan

Deze tabel vat de belangrijkste afwijkingen samen. Ze toont dat vooral de gekozen methoden en de uitvoeringsvolgorde veranderden, terwijl de kern van de opdracht hetzelfde bleef.

Onderdeel	Stageplan	Werkelijke uitvoering
Requirements	Voornamelijk analyseren in week 1.	Bleven tijdens de volledige stage evolueren door prototypes, overleg en tests.
Opslag	Een lokale SQLite-database.	Inspecteerbare JSONL-, JSON- en CSV-bestanden met herstel vanuit een sessie-map.
Replay en simulatie	Replay als onderdeel van de applicatie.	Historische data en live demo's als testmiddelen.
Voorspelling	Een AI- of ML-model in week 5.	Een eenvoudig en uitlegbaar model op basis van recente sectortijden.
Tests	Voornamelijk voorzien in week 6.	Doorlopend uitgevoerd met verschillende databronnen en praktijktests.

De oorspronkelijke weekplanning werd dus maar beperkt gevolgd. Dat betekende niet dat de belangrijkste doelstellingen niet gehaald zijn. Timingdata werd lokaal opgeslagen, geanalyseerd en in een bruikbaar dashboard weergegeven. De geplande methoden voor opslag, replay en voorspelling werden aangepast aan wat binnen zes weken betrouwbaar realiseerbaar en testbaar bleek. De technische gevolgen van deze keuzes worden beschreven in de hoofdstukken over architectuur, dataverwerking en analyse.

Hoofdstuk 3

Technische architectuur, alternatieven en lokale opslag

3.1 Technische architectuur

3.1.1 Electron

De applicatie is gebouwd met Electron en JavaScript. Electron combineert een Chromium-renderer met een Node.js-mainproces. Deze keuze maakte het mogelijk om een webgebaseerde interface te bouwen en tegelijk lokale bestanden en meerdere desktopvensters te beheren. Een bijkomend voordeel was dat dit compatibel was voor beide MacOS en Windows wat nodig was aangezien de app op Windows moest kunnen draaien en de ontwikkeling op MacOS gebeurde.

Het mainproces in `src/main/main.js` beheert de levenscyclus van de applicatie, instellingen, vensters, polling, opslag en IPC-handlers. Een verborgen livevenster laadt de echte timingwebsite. Een extractiescript wordt in die pagina uitgevoerd om tabelkoppen, cellen en sessie-informatie uit de gerenderde DOM te lezen.

3.1.2 Proces- en modulegrenzen

De code is bewust in vier zones verdeeld:

- **src/main:** Electron-specifieke infrastructuur, vensters, updates en bestands-I/O.
- **src/shared:** pure domeinfuncties voor parsing, analyse, voorspelling, pitstops en view-modellen.
- **src/renderer:** HTML, CSS en code die reeds berekende data op het scherm zet.
- **tests:** alle testfiles om domeinfuncties als ook renderer en main functionaliteiten grondig te testen.

De scheiding tussen renderer en domeinlogica was een belangrijke verbetering tijdens deze stage. Vroege versies bevatten berekeningen in de UI-code. Daardoor was het moeilijk om te bewijzen dat een waarde correct was en konden verschillende schermen dezelfde berekening anders uitvoeren. In de huidige architectuur leveren modules zoals `lapAnalytics.js`, `classBattle.js`, `lapPrediction.js` en `pitstopPlanner.js` volledige resultaten. De renderer beperkt zich tot presentatie en gebruikersinteractie.

3.1.3 Vensterarchitectuur

Er is één hoofdvenster^{A.4} voor de primaire wagen. Wanneer de gebruiker via de setup een tweede of derde wagen toevoegt, opent voor elke extra wagen een dashboardvenster met dezelfde renderer en een andere wagenparameter. Alle vensters ontvangen dezelfde collector-status. De timingwebsite wordt dus slechts eenmaal geladen en bevraagd.

Grafieken worden in afzonderlijke vensters geopend.?? Ook zij gebruiken de reeds opgeslagen lap history en starten geen eigen collector. Deze aanpak houdt het hoofdscherm compact en maakt het mogelijk om analysevensters alleen te openen wanneer ze nodig zijn.

3.1.4 Datastroom

Elke poll doorloopt dezelfde stappen:

1. Het extractiescript leest tabellen en sessievelden uit de livepagina.
2. De parser herkent bruikbare headers en zet elke rij om naar canonieke velden.
3. De storage-normalizer voegt provider- en sessiecontext toe.
4. De lap history en actuele rijen worden aan de analysemodules doorgegeven.
5. Voor elke gevolgde wagen worden dashboardanalyse, voorspelling en pitstopplan berekend.
6. De volledige collectorstatus wordt opgeslagen en via IPC naar alle vensters gestuurd.

De UI en de berekeningen worden dus gevoed door dezelfde consistente snapshot. Dit voorkomt dat een grafiek bijvoorbeeld met gegevens van een andere poll rekent dan het hoofdscherm.

3.1.5 Instellingen en configuratie

Instellingen worden als JSON in de Electron user-datafolder bewaard. Ze omvatten timing-URL, gevolgde wagens, modus, opslagmap, referentietijden en pitstopregels. Referentietijden zijn afzonderlijk per sessiemodus opgeslagen, zodat een qualifyingreferentie een racereferentie niet overschrijft. Circuitprofielen voor Spa, Zolder en Assen bevatten standaardwaarden voor de relevante pitlengte en FCY-snelheid. In de pitstopsetup kunnen deze waarden per evenement worden overschreven wanneer het reglement afwijkt.

3.2 Afweging van alternatieve oplossingen

3.2.1 Webapplicatie

Een centrale webapplicatie zou updates en toegang voor meerdere gebruikers vereenvoudigen, maar vereist extra hosting, authenticatie en een betrouwbare netwerkverbinding. Een lokale browserapplicatie vermijdt die afhankelijkheden, maar biedt minder ondersteuning voor bestandsbeheer en meerdere vensters.

Electron maakte één gedeelde implementatie voor macOS en Windows mogelijk en vereenvoudigde de ontwikkeling en distributie. Een native Windowsapplicatie in C# zou nog efficiënter kunnen zijn, maar dat was voor dit prototype minder geschikt door de ontwikkeling op verschillende platformen.

3.2.2 SQLite

SQLite was aanvankelijk de voor de hand liggende keuze. Het biedt transacties, indexes en krachtige queries zonder een afzonderlijke server. Voor de uiteindelijke schaal bleken bestanden echter voordelen te hebben. Een gebruiker kan een CSV direct openen, JSONL kan append-only worden geschreven en een beschadigde laatste regel tast niet noodzakelijk de volledige historiek aan. Ook export en debugging zijn eenvoudiger. Het nadeel is dat relaties en constraints in de applicatiecode worden beheerd en analyses over meerdere evenementen minder efficiënt zijn. Voor een toekomstig centraal racearchief zou SQLite of PostgreSQL daarom geschikter zijn.

3.2.3 Herberekende statistiek

Een gemiddelde kan efficiënt worden bijgewerkt met een teller en som. Voor een lange race lijkt dit aantrekkelijk. In dit project zijn ronden echter niet altijd definitief geldig op het eerste moment dat ze verschijnen. Pitstatus, sectorvlaggen of correcties kunnen de selectie beïnvloeden. Een gecachet gemiddelde zou dan teruggedraaid moeten worden met exact dezelfde eerdere context. Daarom wordt de lap history als bron gebruikt en worden statistieken opnieuw uit de gefilterde relevante ronden berekend. Met enkele duizenden records en pure JavaScriptfuncties blijft dit snel genoeg. De laatst berekende samenvatting wordt wel opgeslagen voor inspectie. Pas wanneer metingen aantonen dat herberekenen een werkelijk prestatieprobleem vormt, is een incrementele cache met invalidatieregels verantwoord.

3.2.4 Machine learning model

Een regressiemodel, random forest of neuraal netwerk zou meerdere features kunnen combineren: piloot, bandenslijtage, brandstofniveau, verkeer, temperatuur en weersomstandigheden. De timingpagina levert veel van die inputs niet. De beschikbare trainingsdata is bovendien beperkt en bevat veranderende circuits en omstandigheden. Een complex model zou daardoor snel overfitten en moeilijk uitlegbaar zijn.

3.3 Dataverwerking en lokale opslag

De betrouwbaarheid van het dashboard hangt af van een consistente verwerking en opslag van de ontvangen timingdata. Gegevens van verschillende timingproviders worden daarom eerst omgezet naar één intern formaat en vervolgens lokaal bewaard. Zo kunnen analyses steeds dezelfde datastructuur gebruiken en blijft de racehistoriek ook beschikbaar na een onderbreking of herstart.

3.3.1 Uniforme parser

De verschillende timingproviders gebruiken verschillende labels en representaties. De parser begint daarom met het opschonen van witruimte en het omzetten van headers naar een standaardvorm. Varianten worden gekoppeld aan interne sleutels voor positie, startnummer, klasse, PIC, team, piloot, aantal ronden, gap, interval, sectoren, laatste tijd, beste tijd en pitinformatie.

Tijdswaarden worden zo vroeg mogelijk naar milliseconden omgezet. De tekst `2:05.073` wordt intern bijvoorbeeld `125073 milliseconden`. Formatteringsfuncties zetten dit pas bij presentatie opnieuw om. Dit maakt optellen, gemiddeldes nemen, vergelijken en testen eenvoudiger.

3.3.2 Vlagstatus en pitfasen

Voor elke sector wordt de vlagstatus vastgelegd op het moment dat de sector beschikbaar wordt. Dit detail is nodig voor het scenario waarin sector 1 groen werd gereden en tijdens sector 2 een FCY begint. De volledige ronde is dan niet representatief voor een rondetijd-gemiddelde, maar sector 1 kan nog geldig zijn voor het gemiddelde en de beste waarde van sector 1. De analyselaag annotateert ook pitfasen. Veranderingen in pitteller en pitstatus helpen inlaps en outlaps identificeren. Deze ronden blijven deel van de historische werkelijkheid, maar worden niet gebruikt voor bijvoorbeeld gemiddeldes te berekenen. Dit voorkomt dat een trage pitronde het gemiddelde van een piloot onnodig verslechtert.

3.3.3 Bestandsformaten

De belangrijkste bestanden zijn:

- `lap_history.jsonl`: append-only historiek met 1 JSON-object per opgeslagen ronde
- `lap_history.csv`: dezelfde rondehistoriek in tabelvorm voor manuele inspectie of spreadsheetanalyse
- `latest_live_rows.json` en `latest_live_rows.csv`: laatste volledige timingsnapshot

- `session_metadata.json`: sessiecontext, gevolgde wagen(s), timingprovider en instellingen voor pitstops
- `analytics_summary.json`: laatst berekende statistieken voor het dashboard en de grafieken
- `lap_prediction.json` en `lap_prediction_car-<nummer>.json`: actuele rondevoorspelling per gevolgde wagen
- `pitstop_plan.json` en `pitstop_plan_car-<nummer>.json`: actuele pitstopplanning per gevolgde wagen
- `gap_state.json` en `gap_history.jsonl`: laatst bevestigde gapinformatie en gap-historiek
- `stint_state.json`: actuele stintinformatie per wagen en piloot
- `track_condition_state.json` en `track_condition_events.jsonl`: huidige baanconditie en wijzigingen doorheen de sessie

JSONL is geschikt voor rondes omdat een nieuwe ronde kan worden toegevoegd zonder het volledige bestand te herschrijven. Tegelijk blijft elke regel leesbaar en herstelbaar. Samenvattingsbestanden worden wel overschreven: ze vertegenwoordigen de actuele afgeleide toestand en kunnen indien nodig opnieuw uit de lap history worden berekend.

Hoofdstuk 4

Analyse, voorspelling en strategie

De lokaal opgeslagen timingdata wordt door de analyselaag omgezet in informatie die de race-engineer tijdens een sessie kan gebruiken. Daarbij worden alleen representatieve rondes en sectoren meegenomen. Op basis daarvan berekent de applicatie statistieken, voorspellingen, klassevergelijkingen en pitstopprojecties die strategische beslissingen ondersteunen.

4.1 Ronde- en pilootstatistieken

Voor een reeks geldige rondes worden laatste tijd, beste tijd, gemiddelde tijd, sectorgemiddelden en beste sectoren bepaald. Pilootstatistieken worden per combinatie van wagen en piloot opgebouwd. De beste piloot van het eigen team is niet hardcoded: wanneer de actuele piloot na zijn stint sneller blijkt, kan hij de vorige referentiepiloot vervangen.

In racemodus vergelijkt het dashboard onder meer:

- beste teamtijd met de laatste ronde van de huidige piloot
- beste teamtijd met de beste ronde van de huidige piloot
- gemiddeld tempo van de beste piloot met dat van de huidige piloot
- tempo van de beste wagen in de klasse met de eigen actuele stint

- tempo van een vrij gekozen klassewagen met de eigen actuele stint

In qualifyingmodus zijn gemiddelden minder relevant. Daar gebruikt de app vooral beste en laatste geldige ronde van teamgenoten en concurrenten. Practice legt de nadruk op referenties en consistente vergelijking zonder pitstopstrategie.

4.2 Voorspelling van de actuele ronde

De rondevoorspelling is bewust eenvoudig en uitlegbaar. Voor de huidige piloot worden per sector maximaal de laatste 5 geldige sectortijden geselecteerd. Na sector 1 is de voorspelling:

$$\hat{T}_{lap} = T_{S1,actueel} + \bar{T}_{S2,recent} + \bar{T}_{S3,recent}.$$

Na sector 2 worden beide actuele sectoren gebruikt:

$$\hat{T}_{lap} = T_{S1,actueel} + T_{S2,actueel} + \bar{T}_{S3,recent}.$$

Na sector 3 wordt de voorspelling niet vervangen door de echte rondetijd. Zo blijft zichtbaar wat na sector 2 voorspeld was. Tussen de finishlijn en de nieuwe sector 1 toont het dashboard de afwijking tussen voorspelling en gerealiseerde ronde. Dit geeft onmiddellijke feedback over de kwaliteit van de voorspellingen.

Voor een nieuwe piloot is na zijn eerste geldige volledige ronde voldoende basisdata beschikbaar om vanaf sector 1 van zijn tweede ronde een eerste voorspelling te maken. Wanneer een piloot eerder al een stint reed, wordt zijn eigen historische sectordata opnieuw gebruikt.

4.3 Referentietijden

De gebruiker kan een referentierondetijd en drie sectorreferenties ingeven via editknoppen in de vakken. De waarden worden per sessiemodus opgeslagen. `normReference.js` berekent de delta en deelt de toestand in als veilig, nabij of kritisch. Deze grenswaarden kunnen ook aangepast worden. Het vak voor predicted lap time krijgt een duidelijke groene, oranje of rode achtergrond. De ideal time is de som van de beste sectoren van de wagen en krijgt eveneens een status. Bij elke sectorreferentie worden twee marges getoond: die van de laatste sector en die van de beste sector. Hierdoor ziet de engineer zowel de actuele uitvoering als het theoretische risico.

4.4 Vergelijkingen binnen de klasse

Een tijdsverschil tussen twee klassewagens mag niet worden afgeleid door alleen hun zichtbare klassepositie te vergelijken. Er kunnen wagens uit andere klassen tussen staan. Wanneer wagens in dezelfde ronde zitten, combineert de applicatie de actuele gap met het verschil tussen recente gemiddelden. Zo ontstaat een indicatie van het aantal ronden of de tijd tot een mogelijke inhaalactie.

4.5 Pitstopplanner

De pitstopplanner gebruikt configureerbare regels voor totale raceduur, gesloten pitwindow tijd aan de start en einde van de race, cooldown na een geldige stop, aantal verplichte stops en geschatte pitduur. De balk toont rode en groene zones en bewaart reeds verstreken rode cooldownsegmenten. Alleen stops die binnen een geldig groen venster plaatsvinden, tellen mee voor het verplichte aantal.

De planner berekent ook het laatste veilige moment waarop nog voldoende verplichte stops met cooldown kunnen plaatsvinden. De pitduur blijft tijdens de race aanpasbaar, omdat een bandenwissel en een pilotenwissel verschillende tijden kunnen vragen.

Voor de projectie na een pitstop worden de relatieve verschillen in de algemene rangschikking opgebouwd totdat de geschatte pit loss is overschreden. Vervolgens wordt bepaald tussen welke klassewagens de eigen wagen zou uitkomen.

4.6 Full Course Yellow

Onder FCY rijdt het veld met een opgelegde snelheid en kan de relatieve pit loss kleiner zijn. Circuitprofielen bevatten daarom de afstand die een niet-pittende wagen tussen pit entry en pit exit over het gewone circuit aflegt en de FCY-snelheid. De tijd op dat traject is:

$$T_{track,FCY} = \frac{d_{track}}{v_{FCY}/3,6}.$$

De geschatte netto pit loss vergelijkt deze trajecttijd en de gap met de ingestelde pitduur. Hierdoor kan er ook tijdens een FCY een juiste inschatting gemaakt worden van de juiste pit loss en de positie in het veld na een pitstop.

4.7 Automatische rapportage

Het idee om automatische PDF-rapporten toe te voegen ontstond tijdens de praktijktest in Spa. Na zijn stint vroeg een piloot hoe zijn stint verlopen was. Op dat moment kon ik vooral zeggen dat hij ronde na ronde iets sneller werd, maar ik had nog geen duidelijk overzicht van waar die verbetering vandaan kwam: welke sectoren beter werden, welke ronden representatief waren, hoe constant de stint was en hoe hij het deed in vergelijking met teamgenoten of klassewagens. Ook Jan vroeg na de race om een verslag van de volledige wedstrijd. Dat maakte duidelijk dat deze rapporten niet alleen nuttig zijn voor onmiddellijke feedback aan de piloot, maar ook voor latere evaluatie.

Naast het live dashboard kan de applicatie daarom PDF-rapporten genereren op basis van de lokaal opgeslagen racehistoriek. Per pilootstint wordt een apart document gemaakt met de rondetijden, sectoren, sectorgemiddelden, beste en traagste geldige ronden, uitgesloten ronden en vergelijkingen met teamgenoten en klassewagens. De rapporten gebruiken dezelfde centrale selectieregels als het dashboard: FCY-, Safety-Car-, pit-in-, outlap- en duidelijke outlierronden blijven bewaard, maar worden niet gebruikt voor gemiddelden te berekenen.

Voor de volledige race wordt ook een overzichtspagina gegenereerd. Deze bundelt onder andere het aantal gereden ronden, geldige ronden, gemiddelde rondetijd, beste ronde, sectorgemiddelden, pitstopinformatie en vergelijking tussen piloten. Hierdoor kan de engineer na de sessie sneller zien waar tijd werd gewonnen of verloren, welke stint het meest constant was en hoe

sterk de pitstops afweken van de verwachte pitduur. De PDF's zijn dus geen aparte databron, maar een leesbare samenvatting van dezelfde JSON-, JSONL- en CSV-bestanden die tijdens de race werden opgebouwd. Een voorbeeld van een automatisch gegenereerde race-PDF op basis van de Spa-data is opgenomen in de appendix in Bijlage B.

Hoofdstuk 5

Praktijktesten op het circuit

5.1 Context van het eerste raceweekend

Het dashboard werd tijdens een volledig raceweekend in Spa gebruikt met "RIS Timing" als timingprovider. Dit was de eerste en enige officiële praktijktest tijdens een raceweekend vóór de *24 Hours of Zolder*. Vrijdag kon de vrije training succesvol worden gevolgd en werd de ondersteuning voor een tweede timingprovider in een echte omgeving getest. De kwalificatie verliep anders dan gepland: door motorschade kon de voorziene wagen niet verder worden gebruikt. Het team bouwde gedurende de nacht een ouder Mazda MX-5-model op om toch aan de race deel te nemen. Deze wagen had ongeveer 50 pk minder vermogen en reed op oude banden uit noodzaak. De omstandigheden waren daardoor niet geschikt om de absolute prestaties van de oorspronkelijk geplande wagen te beoordelen, maar leverden wel waardevolle circuitervaring voor de nieuwe piloot en een realistische stresstest voor de applicatie.

Deze test was relevant met het oog op de 24 uur van Zolder, waarvoor het dashboard meerdere teams moet kunnen volgen. Samen met Jan ondersteunde ik enkele strategische afwegingen. Daarnaast gaf hij gerichte feedback over de gebruikersinterface, die ik na het raceweekend meteen kon verwerken.

5.2 Uitgevoerde tests en verzamelde data

Tijdens practice, pre-qualifying, qualifying en race werden afzonderlijke sessiemappen bijgehouden. De opgeslagen data bestond uit rondehistoriek, analytics, rondevoorspellingen, pitstopplannen en sessiemetadaten. Tijdens de race werden onder andere de volgende onderdelen gebruikt:

- parsing van RIS Timing, inclusief sessiegegevens, pilootnamen, sectoren, GAP en INT
- langdurige opslag en hervatting van rondehistoriek
- piloot-, team- en klassevergelijkingen
- rondevoorspelling en referentietijden
- gaps naar klassewagens en catchindicaties
- verplichte pitstops en de voorspelde positie na een pitstop
- grafieken en meerdere gevolgde wagens

De test bewees dat de volledige keten gedurende een echte sessie data kon verzamelen en bewaren. Ze bracht tegelijk fouten aan het licht die niet voldoende zichtbaar waren in demo's en zelfgeschreven unit tests. De Spa-data wordt daarom niet alleen als resultaat bewaard, maar ook als regressiedataset voor volgende versies.

5.3 Belangrijkste bevindingen

5.3.1 Vlagcontext en geldige rondes

De belangrijkste datakwaliteitsfout was het moment waarop FCY-status aan een ronde en sector wordt gekoppeld. Wanneer FCY begint terwijl een wagen in sector 2 rijdt, moet sector 2 onmiddellijk als geneutraliseerd worden gemarkeerd. Sector 1 blijft geldig wanneer die volledig onder groen werd gereden maar de volledige ronde mag niet meer meetellen als representatieve rondetijd.

Een concreet voorbeeld kwam in ronde 2. In `lap_history.jsonl` staat de afgeronde ronde met `lapFlag = Green flag`, terwijl de opgeslagen sectorflags FCY aangaven. De sectoren werden hierdoor al uitgesloten, maar de rondecontext is intern niet consistent. Daarnaast moeten pit-in- en outlaps en duidelijke tempo-outliers door exact dezelfde centrale selectieregel worden uitgesloten. De test toonde dus dat een globale sessieflag op het moment van afronden niet volstaat maar dat de applicatie ook de sector waarin iedere gevolgde wagen zich bevindt moet onthouden wanneer de vlag verandert.

5.3.2 Gemiddelden en vergelijkingseenheden

De getoonde gemiddelden waren op enkele momenten hoger dan een handmatige controle deed verwachten. Waarschijnlijke oorzaken zijn onvolledige vlagcontext en pitgerelateerde rondes die niet overal identiek gefilterd werden. Bovendien kon dezelfde wagen tegelijk als beste wagen in de klasse en als geselecteerde vergelijkingswagen verschijnen, terwijl beide vakken toch een ander gemiddelde toonden. Dat verschil ontstaat wanneer het ene vak een volledige wagenhistoriek gebruikt en het andere een actuele piloot of stint, zonder dit duidelijk te benoemen. De gewenste oplossing is één volledig analyticsdocument met benoemde statistieken en selectiecriteria. Dashboard, grafieken en rapportage moeten dezelfde waarden uit dat document gebruiken.

5.3.3 Gaps en pitstopprojectie

Gaps naar de voorliggende en achterliggende klassewagens veranderden tijdens de race te sterk om operationeel betrouwbaar te zijn. RIS Timing werkte met GAP tot de leider en INT tot de vorige wagen dus tussenliggende wagens uit andere klassen moesten in de intervalketen worden opgenomen. Rondeachterstanden maakten een directe omzetting bovendien grof en gevoelig voor het meetmoment. Voor volgende iteraties is een gapgeheugen nodig. Een gap wordt pas als nieuwe stabiele waarde gepubliceerd wanneer de betrokken wagen een nieuw meetpunt passeert, bij voorkeur de start-finishlijn. De UI moet groot de laatst bevestigde gap tonen, met daaronder het verschil tussen de recentste rondetijden en een afzonderlijke catchtrend. Een wagen die al enige tijd (5 van onze rondes hebben we voor nu gekozen) in de pits staat, moet tijdelijk als niet-actieve tegenstander worden gemarkeerd in plaats van een misleidende catchtijd te krijgen. Dezelfde stabiele gapreeks kan later de pitstopprojectie en een gap-over-timegrafiek voeden.

5.4 Nieuwe requirements uit de praktijktest

De test leverde naast correcties ook drie nieuwe uitbreidingswensen op:

1. Een piloottimer met duur van de huidige stint en totale rijtijd per piloot. Want ook dit is belangrijk tijdens endurance races, piloten mogen niet langer dan een bepaalde periode in de auto zitten volgens de regels. Pilootwissels vormen hier de stintgrens, sessietijd en timestamps leveren de duur.
2. Een scrollbare recente-rondenstrook met rondenummer en kleurstatus: pit-in met rode markering en letter P, outlap rood, FCY/SC geel, snelste eigenronde groen en als deze de snelste ronde van alle auto's in de klasse is deze paars (dit zijn conventies uit alle niveaus in de motorsport). De volledige historie blijft beschikbaar, terwijl standaard slechts de laatste tien tot twintig ronden zichtbaar zijn.
3. Automatische PDF-rapportage per stint en na de race. Een stintdocument bevat ronde- en sectortijden, geldige gemiddelden, vergelijking met teamgenoten en de gapevolutie. Bij een pilootwissel kan automatisch een map zoals bijvoorbeeld `STINT_2_JANSSENS_Robbe` worden gemaakt. Een raceoverzicht combineert de stintresultaten per piloot en voor het team.

Deze rapportage is technisch haalbaar omdat de onderliggende gegevens al lokaal worden opgeslagen. Voor betrouwbare automatische generatie moeten eerst stintdetectie, vlagcontext en centrale analytics eenduidig zijn, anders zou een verzorgd PDF-document dezelfde foutieve selectie reproduceren.

5.5 Prioriteiten na Spa

De vervolgvolgorde wordt bepaald door databetrouwbaarheid, niet door zichtbaarheid in de interface:

1. vlagtransities per wagen en sector correct registreren en alle pacefilters centraliseren
2. één persistent analyticsmodel invoeren
3. stabiele gaps op meetpunten bewaren en gebruiken voor catch- en pitstopprojecties
4. stintdetectie, pilootduur en recente-rondenstrook toevoegen
5. automatische stint- en racerapporten genereren
6. pas daarna de geplande UI-herwerking uitvoeren op basis van dezelfde opgeslagen view-modellen

5.6 Aanvullende tests tussen Spa en Zolder

Omdat Spa de enige officiële praktijktest was, konden nieuwe functies die daarna werden toegevoegd niet meer tijdens een vergelijkbaar raceweekend worden getest. Dat maakte het moeilijk om met zekerheid te bepalen of nieuwe berekeningen en interfacewijzigingen ook onder echte racedruk correct zouden blijven werken. De opgeslagen Spa-data bleef bruikbaar voor

regressietests, maar bevatte uiteraard niet elke situatie die tijdens een 24-uursrace kan voorkomen. Om toch voortdurend met veranderende timingdata te testen, gebruikte ik daarom veelvuldig de live demo van GetRaceResults¹. Die demo maakte het mogelijk om de parser, de live updates en de dashboardweergave herhaaldelijk te controleren zonder op een volgend officieel raceweekend te moeten wachten. Ze vervangde echter geen volledige praktijktest want de demo bood uiteraard niet dezelfde combinatie van meerdere teams, pitstops, neutralisaties, pilootwissels en operationele tijdsdruk als tijdens een echte endurancewedstrijd. Daarnaast werden gerichte unit tests gebruikt om zoveel mogelijk edge cases af te dekken. Daarbij werd onder meer gecontroleerd hoe de applicatie omgaat met ontbrekende of onvolledige sectoren, pit-in- en outlaps, vlagovergangen, rondeachterstanden, wisselende piloten en tegenstanders die langdurig in de pits staan. Deze tests vergrootten het vertrouwen in afzonderlijke berekeningen, maar konden niet volledig aantonen hoe alle componenten gedurende 24 uur samen zouden functioneren. Juist daardoor bleef er voor de *24 Hours of Zolder* de nodige spanning want het was tegelijk het einddoel van de stage en de eerste keer dat de nieuwste functies in hun volledige gebruikscontext zouden worden ingezet.

5.7 Het einddoel: *24 Hours of Zolder*

5.7.1 Verwachte meerwaarde tijdens de race

De grootste meerwaarde tijdens de 24 uur van Zolder lag in het combineren van informatie die op de timingpagina verspreid of impliciet aanwezig is. Het dashboard toont niet alleen de actuele positie, maar vertaalt die naar teamgerichte vragen:

- Hoe snel rijdt de huidige piloot tegenover zijn teamgenoten?
- Hoe evolueert het tempo tegenover de beste wagen in de klasse?
- Hoeveel marge is er tegenover de referentietijd?
- Wanneer wordt een pitstop reglementair mogelijk of net riskant?
- Waar komt de wagen vermoedelijk terug op de baan na een pitstop?
- Welke rondes of sectoren zijn representatief genoeg om in gemiddelden te gebruiken?

Deze vragen zijn vooral belangrijk omdat Jan tijdens de race meerdere rollen tegelijk opneemt: strategie, coaching, communicatie en opvolging van het wedstrijdverloop. Een dashboard dat herhalende berekeningen automatisch uitvoert, verkleint de kans dat belangrijke details worden gemist wanneer er veel tegelijk gebeurt. De applicatie neemt geen beslissingen in de plaats van de race-engineer, maar maakt de beschikbare informatie sneller interpreteerbaar.

5.7.2 Wat tijdens de 24 uur getest moest worden

Hoewel de applicatie voor de race klaar was als werkende basis, moest de 24 uur van Zolder nog aantonen hoe goed ze standhield in haar uiteindelijke context. De belangrijkste testpunten waren:

- Langdurige stabiliteit gedurende een volledige 24-uursrace.

¹<https://livetiming.getraceresults.com/demo#screen-results>

- Correcte opvolging van meerdere wagens tegelijk.
- Betrouwbare stintdetectie bij alle pilootwissels.
- Correcte behandeling van FCY, Safety Car, rode vlag en pitgerelateerde rondes.
- Bruikbaarheid van de pitstopplanner wanneer snel beslist moet worden.
- Leesbaarheid van het dashboard in een drukke raceomgeving.
- Nut van de automatische stint- en racerapporten na de wedstrijd.

Vooral de combinatie van een lange looptijd, meerdere pitstops, wisselende wedstrijdsituaties en menselijke werkdruk kon niet volledig met korte tests worden nagebootst. De 24 uur van Zolder was daarom niet alleen het einddoel van de applicatie, maar ook de belangrijkste test van de ontwerpkeuzes.

5.7.3 *24 Hours of Zolder*: het verloop van de race

Van kwalificatie tot finish

Tijdens de vrije trainingen en de kwalificatie op donderdag maakten de piloten kennis met de wagen en werkte de applicatie zonder problemen. Daarna volgde de *Suppole*, een kortere kwalificatiesessie waarin de zes piloten elk een snelle ronde moesten afleggen. Omdat de wagen te laat de pitbox verliet, konden niet alle piloten hun ronde rijden. Daarom besloot het team de sessie niet voort te zetten en moest het achteraan starten.

Ook de race verliep moeizaam. Na ongeveer vijf ronden viel de wagen stil doordat de batterij-aansluiting door trillingen was losgekomen. Kort na deze reparatie werd de Mazda door wagen #70 van de baan geduwd en kwam hij hard in het gras tot stilstand. Robbe Janssen werd met pijn aan zijn ribben en hoofdpijn uit voorzorg naar het ziekenhuis gebracht. Na meer dan een uur repareren kon Bert Longin opnieuw vertrekken om de wagen te controleren. Daarbij bleek ook de behuizing van de versnellingsbak gescheurd te zijn, waardoor de versnellingsbak volledig moest worden vervangen. Ondanks alle tegenslagen bereikte de Mazda uiteindelijk toch de finish. Het gehoopte podium bleef uit, maar na de technische problemen en de zware crash was het uitrijden op zich al een waardevol en leerrijk resultaat.

Evaluatie van het dashboard

Tijdens de trainingen en de reguliere kwalificatie werkte het dashboard zoals verwacht. De timingdata werd correct verwerkt en geanalyseerd. Door de vele problemen met de auto en de lange reparaties kon de applicatie tijdens de race zelf minder intensief worden gebruikt dan oorspronkelijk gehoopt. Het dashboard werd later tijdens de race verder getest met zusterwagen #509. A.5 Ook bij deze tests deden zich geen problemen voor. Deze tests bevestigden dat de applicatie stabiel genoeg is om gedurende de rest van het seizoen volledig door het team te worden ingezet.

Hoofdstuk 6

Evaluatie en persoonlijke reflectie

6.1 Evaluatie van het stageplan

Het oorspronkelijke plan legde de nadruk op dataverzameling, lokale opslag, visualisatie en voorspelling. De technische invulling die ik in het stageplan had opgeschreven veranderde echter sterk. SQLite werd vervangen door transparante JSONL-, JSON- en CSV-bestanden. Replay werd uit de eindapp verwijderd en vervangen door een testgerichte simulator. Een complex AI/ML-traject maakte plaats voor een simpele en uitlegbare sectorvoorspelling. Tegelijk groeide de race-engineeringfunctionaliteit met meerdere sessiemodi, pitstopplanning, verschillende providerondersteuning, meerdere dashboards en grafieken.

Die verschuiving was nodig want binnen zes weken was het belangrijker om de volledige live keten betrouwbaar te maken dan om een complex model bovenop onzekere data te bouwen. De moeilijkste problemen zaten niet in de rekenkundige formules, maar in de betekenis en kwaliteit van timingvelden. Zonder correcte providerinterpretatie zou een geavanceerd model alleen preciezere foutieve resultaten produceren.

6.2 Zelfstandig werken

De technische uitvoering gebeurde grotendeels zelfstandig. Zelfstandigheid betekende ook beslissen wanneer een vraag technisch kon worden opgelost en wanneer domeinfeedback nodig was. De betekenis van een referentietijdregel of geldige pitstop kan niet uit code worden afgeleid. In zulke gevallen was overleg met de opdrachtgever noodzakelijk voordat verdere implementatie kon gebeuren.

6.3 Communicatie en requirementsanalyse

Het meermaals overleggen met de klant was een centraal deel van de stage. Presentaties met uitleg en screenshots van wat er was veranderd en concrete scenario's bleken effectiever dan abstracte technische uitleg. Een vraag als "Waar staan we na een pitstop van 75 seconden?" maakte onmiddellijk zichtbaar welke data en aannames ontbraken. De implementatie kon vervolgens worden uitgelegd in termen van opeenvolgende verschillen, rondeachterstanden en klasseposities.

Ik leerde requirements niet als vaste lijst te behandelen. De opdrachtgever kon pas na gebruik beoordelen of een veld groot genoeg was, of een kleur voldoende opviel en of een vergelijking tijdens een race werkelijk nuttig was. Tegelijk moest ik ervoor zorgen dat elke wijziging een duidelijke betekenis hield en niet alleen visueel mogelijk werd.

6.4 Software-engineeringvaardigheden

De stage bracht verschillende onderdelen uit de opleiding samen: modulariteit, datastructuren, foutafhandeling, testontwerp en user-interfaceontwerp. Vooral *separation of concerns* werd

praktisch belangrijk. Pure shared modules bleken niet alleen beter testbaar, maar maakten ook meerdere dashboards en grafiekvensters mogelijk zonder logica te kopiëren. Het project maakte ook het verschil duidelijk tussen syntaxis en semantiek. Een parser kan een tekst perfect lezen en toch een fout resultaat produceren wanneer bijvoorbeeld een gap van 5L als vijf seconden wordt behandeld. Betrouwbare software vereist daarom domeinconstraints en tests met betekenisvolle voorbeelden.

6.5 Onverwachte competenties

Naast de competenties die ik vooraf had bedacht te verbeteren, ontwikkelde ik tijdens de stage ook vaardigheden waarvan ik niet van verwachtte dat ze zo belangrijk zouden worden.

Een eerste onverwachte competentie was het kritisch beoordelen van externe data. Aan het begin ging ik ervan uit dat waarden op een timingpagina rechtstreeks gebruikt konden worden wanneer ze correct werden ingelezen. Tijdens het ontwikkelen bleek dat een waarde syntactisch geldig kan zijn, maar toch een andere betekenis kan hebben dan verwacht. Ik leerde daarom niet alleen controleren of data aanwezig is, maar ook of de eenheid, context en betekenis ervan correct zijn.

Als tweede leerde ik meer denken in operationele risico's. Software die tijdens een race wordt gebruikt, moet niet alleen correct werken in normale omstandigheden. Ook een wegvallende timingpagina, een foutieve sluiting, ontbrekende sectorinformatie of een veranderende vlag-status moet voorspelbaar en correct worden afgehandeld. Hierdoor begon ik betrouwbaarheid en herstelbaarheid belangrijker te vinden dan het zo snel mogelijk nieuwe functies proberen toe te voegen.

Als laatste kreeg ik meer inzicht in de volledige oplevering van een softwareproduct. De applicatie moest ook op verschillende besturingssystemen kunnen worden verpakt, geïnstalleerd en bijgewerkt. Daarnaast moesten instellingen en opgeslagen racedata begrijpelijk blijven voor iemand die de interne implementatie niet kent. Deze ervaring leerde mij om niet alleen als ontwikkelaar naar afzonderlijke functies te kijken, maar ook verantwoordelijkheid te nemen voor het uiteindelijke gebruik van het volledige product.

6.6 Wat beter kon

Hoewel het project uiteindelijk een bruikbare applicatie opleverde, zijn er verschillende punten die achteraf beter of vroeger aangepakt hadden kunnen worden.

1. Een formele provider-specificatie aan het begin had sommige interpretatiefouten kunnen voorkomen. Vooral het verschil tussen **GAP**, **DIFF** en **INT** bleek afhankelijk van de timingprovider en zelfs van de sessie. Omdat de feeds vooral via demo's en screenshots beschikbaar waren, werden sommige aannames pas tijdens echte tests ontdekt.
2. Het concept van "geldige tempo-data" had vanaf het begin als aparte domeinregel moeten bestaan. Safety-car-ronden, FCY-ronden, rode vlag, pit-inlaps, outlaps en extreme outliers kwamen eerst op verschillende plaatsen in de code terug. Daardoor ontstond het risico dat een ronde in de ene berekening wel werd uitgesloten en in een andere niet.
3. De testdata had vroeger dichter bij echte racesituaties mogen liggen. Veel unittests werkten met kleine, nette voorbeelden, terwijl echte timingdata ontbrekende sectoren, veranderende rijnamen, pitstatussen en rondeachterstanden bevat. De Spa-data werd

pas later een belangrijke regressiebron. Later werd duidelijk dat racehistoriek op de timingwebsite beschikbaar bleef. Als dat vroeger bekend was geweest, had ik sneller met echte racedata kunnen testen.

4. De ondersteuning voor meerdere gevolgde auto's had vroeger expliciet in het ontwerp kunnen zitten. De basisdata hoeft maar één keer opgehaald te worden, maar dashboards, referentietijden, instellingen en rapporten zijn wel per auto verschillend. Dat onderscheid werd pas later volledig duidelijk.
5. De configuratie van racespecifieke instellingen had vanaf het begin duidelijker gescheiden kunnen worden van live aanpasbare instellingen. Totale raceduur, verplichte pitstops en circuitinformatie veranderen normaal niet tijdens een race, terwijl pitstoptijd, referentietijden en track condition wel kunnen veranderen. Die scheiding is uiteindelijk toegevoegd, maar had eerder voor minder verwarring gezorgd.

Hoofdstuk 7

Relatie met de opleiding

7.1 Objectgericht programmeren: *H01P1A*

Objectgericht programmeren uit de bachelor, dit vak noemt nu "modulair programmeren", was vooral belangrijk voor het denken in duidelijke modules met een afgebakende interface. Hoewel de applicatie in JavaScript en niet in Java geschreven is, bleef hetzelfde principe gelden: elk onderdeel moet een duidelijk doel hebben en via een beperkte API gebruikt kunnen worden. Daardoor kan een module zoals de pitstopplanner of de lapanalyse getest worden zonder de volledige Electron-app op te starten. Dit maakt het ook makkelijker om de applicatie uit te breiden met nieuwe functionaliteit, zoals een extra dashboard of een nieuwe timingprovider.

Ook het idee van abstractie kwam sterk terug. Timingproviders leveren niet exact dezelfde kolommen of betekenissen op, maar de rest van de applicatie mag daar zo weinig mogelijk (of liefst geen) last van hebben. Daarom worden live rijen eerst genormaliseerd naar een uniform model. De verdere berekeningen werken dan op die abstractie in plaats van rechtstreeks op de HTML of CSV-structuur van een specifieke website. De aandacht uit het vak voor randgevallen, specificaties en systematisch testen was daarbij nuttig, zeker omdat veel bugs net ontstonden door uitzonderlijke situaties zoals ontbrekende sectoren, rondeachterstanden, safety-car-ronden of pit-in/outlaps.

7.2 Software-ontwerp: *G0Q40C*

Dit bachelorvak was heel herkenbaar in de dagelijkse ontwikkeling van het dashboard. In dit vak lag de nadruk op objectgerichte ontwerpprincipes, ontwerppatronen, refactoring en testbaarheid. Tijdens de stage kwam dit vooral terug in de manier waarop de berekeningen uit de gebruikersinterface werden gehaald. De renderer mocht uiteindelijk enkel data tonen en interacties doorgeven, terwijl de echte domeinlogica in aparte modules terechtkwam. Door deze scheiding kon de logica onafhankelijk van de Electron-app afzonderlijk getest worden.

Ook het iteratieve aspect van het vak sloot goed aan bij dit project. De applicatie groeide

door feedback van de opdrachtgever en door praktijktesten. Regelmatig moest bestaande code worden herwerkt omdat een regel anders bleek te zijn dan eerst gedacht, bijvoorbeeld bij pitstops, rondeachterstanden of een andere timing-site. De principes rond refactoring en testen hielpen om zulke aanpassingen door te voeren zonder telkens het volledige systeem onoverzichtelijker te maken.

7.3 Software Architecture: *H07Z9B*

Dit vervolgvak op software-ontwerp was nuttig omdat dit project al snel meer werd dan een verzameling losse scripts. De applicatie moest live timingdata ophalen, die data normaliseren, lokaal opslaan, berekeningen uitvoeren, een dashboard tonen, grafieken openen en achteraf PDF-rapporten genereren. Daarbij waren niet alleen de functionele eisen belangrijk, maar vooral ook de kwaliteitseisen: betrouwbaarheid tijdens een race, uitbreidbaarheid voor nieuwe timingproviders, leesbare opslag en herstel na een onderbreking.

Het vak hielp me vooral om vooruit te denken over de architectuur van de applicatie en niet zomaar code te schrijven zonder idee. De keuze om parsing, opslag, analyses, pitstoplogica, voorspellingen en renderer-code van elkaar te scheiden komt rechtstreeks voort uit het idee dat architectuur de verdere levenscyclus van software bepaalt. Ook de trade-off tussen eenvoudige JSON/CSV-bestanden en een database werd niet enkel technisch bekeken, maar als architecturale keuze. De inspecteerbaarheid en herstelbaarheid waren voor deze toepassing belangrijker dan maximale databankfunctionaliteit.

Nog een belangrijke les die ik geleerd heb in dit vak is dat het bijna onmogelijk is om voor elke mogelijke edge case voorbereid te zijn. Daarom is het belangrijk om een systeem te ontwerpen dat makkelijk aanpasbaar is en dat je makkelijk kan testen. Dit was ook een van de redenen waarom ik gekozen heb om de data lokaal op te slaan in plaats van in een database. Zo kon ik makkelijk de data inspecteren en testen zonder dat ik telkens een database moest opstarten.

7.4 Fundamentals of Human-Machine Interaction: *H0V14A*

De raceomgeving stelt specifieke HCI-eisen. Een gebruiker heeft geen tijd om meerdere menu's te doorlopen of lange uitleg te lezen. Informatiehiërarchie, stabiele boxafmetingen en kleur-statussen moesten daarom samen worden ontworpen. Zo bleek het verschil tussen een dunne rode rand en een volledig lichtrood predictionvak operationeel relevant: de tweede variant trekt sneller aandacht.

Tegelijk mag kleur niet de enige informatiedrager zijn. Labels, delta's en statustekst blijven zichtbaar. De keuze om grafieken in een afzonderlijk venster te plaatsen is eveneens een HCI-afweging. Detailanalyse blijft beschikbaar zonder het primaire racescherm permanent te belasten.

Het vak was ook nuttig omdat het benadrukt dat interfaces moeten vertrekken vanuit gebruikers en taken, niet vanuit de interne datastructuur. De opdrachtgever had tijdens een race vooral nood aan snelle beslissingsinformatie: welke auto wordt gevolgd, wat is de huidige pace, hoeveel marge is er voor een pitstop en welke informatie vraagt onmiddellijk aandacht? Daarom werden veel onderdelen opnieuw ontworpen nadat bleek dat ze visueel te druk waren of tijdens de race niet snel genoeg begrepen werden. De opeenvolgende UI-versies waren dus in feite kleine cycli van ontwerpen, testen, feedback verzamelen en herwerken.

7.5 Bedrijfservaring: Computerwetenschappen: *H04G0A*

Na mijn stage van vorige zomer bij Nyrstar, heb ik gezien hoe iteratieve processen in een echte bedrijfsomgeving te werk gaan. Hier heb ik geleerd om kleine deadlines te zetten voor kleine delen van het project. Door al deze kleine bijdragen ergens bij te houden, hou je een duidelijk overzicht van wat al gedaan is en wat nog moet gebeuren. Daarom heb ik hiervoor Github Project gebruikt. Hiermee hou je automatisch een *Status Board* bij waarin duidelijk te zien is wat er al gebeurd is en wat er nog moet gebeuren. Ook kan je hier makkelijk een *Roadmap* maken om een tijdlijn te geven van wat je wanneer wilt gaan doen. Ik vond dit een zeer handige tool aangezien voor een project als dit heel veel kleine TODO's heeft die als je ze nergens bijhoudt snel kunt vergeten.

Ook heb ik tijdens mijn vorige stage veel geleerd over hoe ik feedback moet vragen en verwerken. Aangezien ik het dashboard niet voor mezelf heb gemaakt, was het zeer belangrijk dat ookal vond ik sommige aanpassingen niet nuttig, luisteren naar wat de klant wilt is in dit geval belangrijker. Maar bij Nyrstar heb ik ook geleerd dat ik mijn eigen input altijd mag geven aan de klant. Zo heb ik een aantal features kunnen toevoegen waar de klant nog niet aan gedacht had maar toch zeer handig vond eens ze er waren. Daarom zou ik iedereen aanmoedigen om beide stages te doen omdat je hier zoveel leert over hoe je in een echte bedrijfsomgeving te werk gaat en hoe je bijvoorbeeld met klanten moet omgaan. Het is een zeer waardevolle ervaring die je niet tijdens de lessen leert.

Hoofdstuk 8

Conclusie

Tijdens deze stage werd een werkende basisversie van het MSTC Race Engineer Dashboard ontwikkeld. De applicatie leest live timingpagina's, normaliseert gegevens van verschillende providers, bewaart een hervatbare racehistoriek en zet die data om in teamgerichte informatie. Het dashboard toont onder meer recente rondes, sectoren, referentietijden, klassevergelijkingen, pitstopinformatie, grafieken en automatische PDF-rapporten.

De belangrijkste technische bijdrage is de volledige keten van ruwe timingdata naar bruikbare race-engineeringinformatie. Rondes en sectoren worden met context opgeslagen, niet-representatieve rondes worden uitgesloten uit tempoanalyses en berekeningen zoals voorspellingen, klassevergelijkingen en pitstopprojecties zitten in aparte modules. Daardoor bleef de software testbaar en uitbreidbaar, ook toen de eisen tijdens de stage sterk evolueerden.

De test in Spa-Francorchamps bevestigde dat de dataketen met RIS Timing in een echte raceomgeving bruikbaar was. Tegelijk kwamen daar precies de details naar boven die alleen live zichtbaar worden: vlagtransities, pit-in- en outlaps, stabiele gaps en de selectie van geldige pace-ronden. De opgeslagen Spa-data kon later gebruikt worden in regressietests, waardoor de applicatie ook na de race betrouwbaar bleef werken.

De *24 Hours of Zolder* was het uiteindelijke einddoel van deze stage. Tijdens de trainingen en de kwalificatie werkte de applicatie zoals verwacht. Door een losgekomen batterijaansluiting, een zware crash en een beschadigde versnellingsbak verliep de race anders dan gepland en kon het dashboard minder intensief worden gebruikt. Tijdens verdere tests met zusterwagen #509 bleef het dashboard zonder problemen functioneren. Hiermee werd bevestigd dat het dashboard stabiel genoeg is om de rest van het seizoen ingezet te worden voor het team.

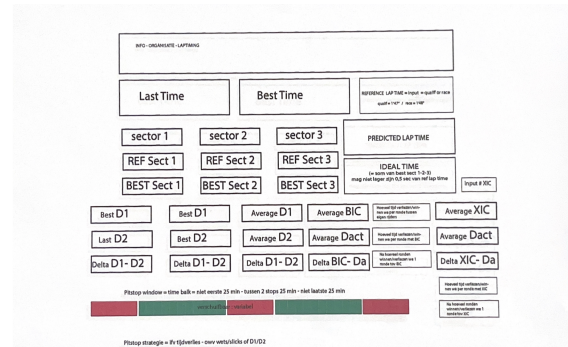
De stage vormde ook persoonlijk een mooie start in de motorsport. Volgend jaar werk ik aan mijn thesis bij *Team WRT*, een van de grootste en meest succesvolle Belgische raceteams. Daar zal ik pitstops analyseren met computer graphics en AI om de pitstops nog verder te optimaliseren. Deze stage toonde al duidelijk waarom dat relevant is: ook in endurancewedstrijden, die de laatste jaren steeds meer op lange sprintraces lijken, kan een trage pitstop een grote invloed hebben op de uiteindelijke uitslag. Ik kijk er enorm naar uit om de kennis die ik in deze stage over motorsport heb opgedaan volgend jaar te gebruiken tijdens mijn thesis.

Bijlage A

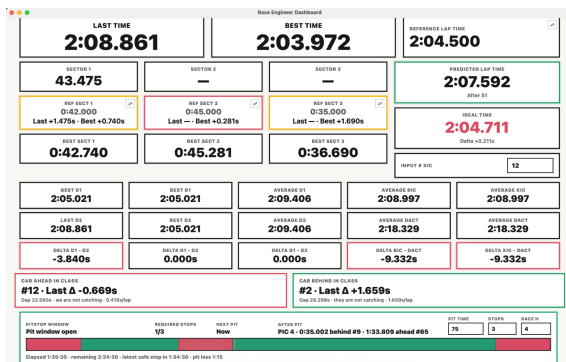
Aanvullende afbeeldingen

P	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
1	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
2	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
3	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
4	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
5	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
6	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
7	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
8	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
9	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
10	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
11	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
12	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
13	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
14	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
15	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT
16	DRIVER	CLASS	PC	#	TEAM INFO	LAP	GAP	45.00	51.00	56.00	LAST	BEST	ETA	PT

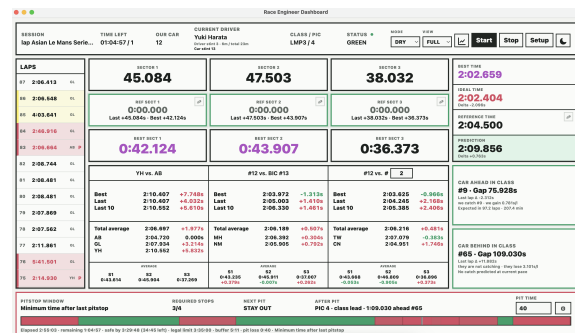
Figuur A.1: Screenshot van de officiële timingpagina tijdens de race in Spa-Francorchamps. De tabel toont actuele ronde- en sectortijden en maakt zichtbaar hoe verschillende klassen door elkaar staan.



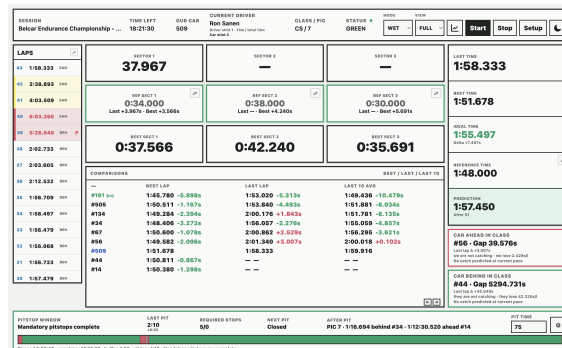
Figuur A.2: Schets van het eerste dashboardontwerp. Deze schets werd door de opdrachtgever gemaakt en vormde de basis voor de eerste werkende versie.



Figuur A.3: Eerste werkende versie van het dashboard, getest tijdens de race in Spa-Francorchamps.



Figuur A.4: Finale versie van het dashboard.



Figuur A.5: Dashboard tijdens de 24h of Zolder.



Figuur A.6: Garage tijdens de 24 Hours of Zolder.



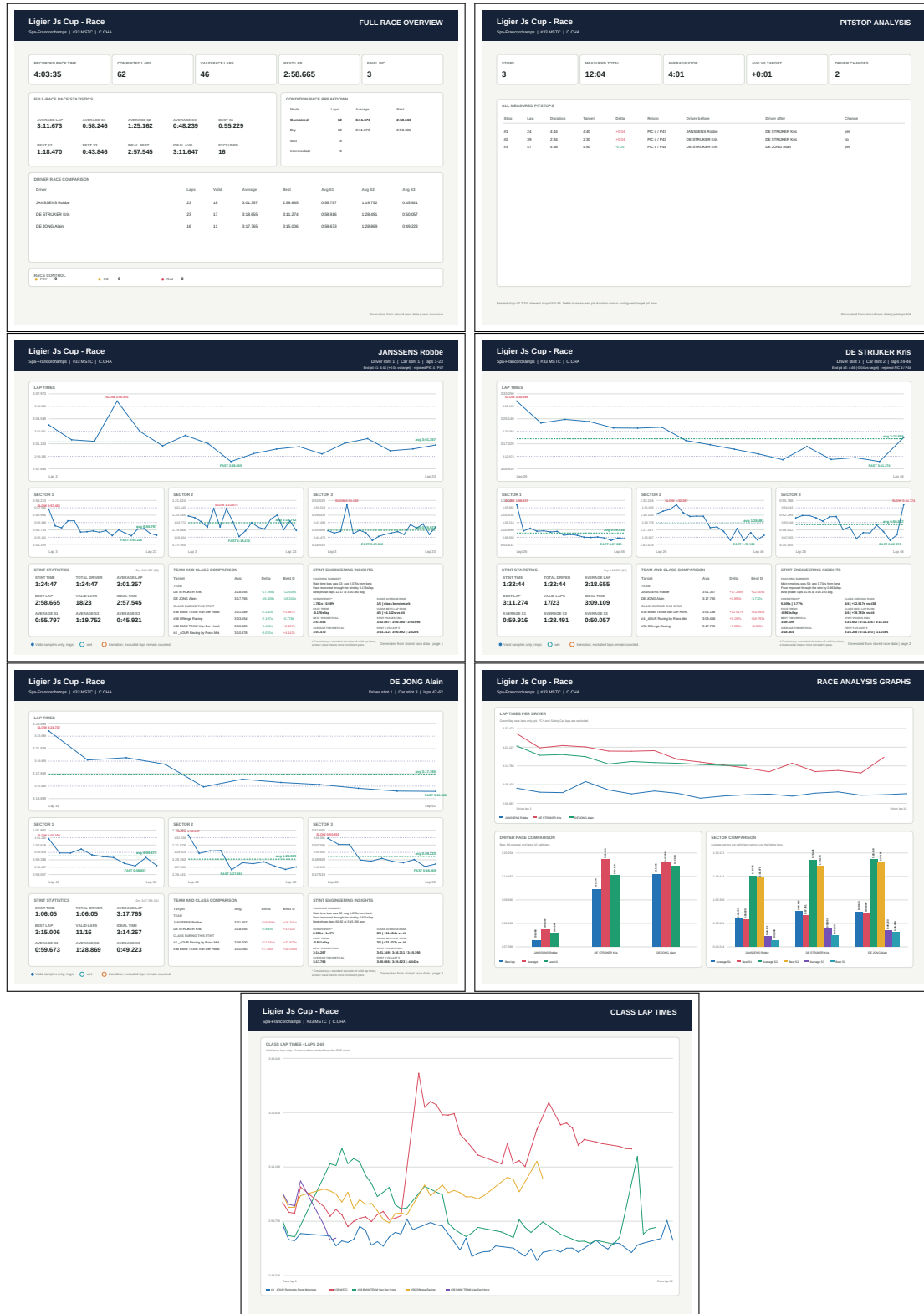
Figuur A.7: Auto na de crash.



Figuur A.8: Eerste reparaties na de crash.

Bijlage B

Voorbeeld van automatisch gegenereerd raceverslag



Afkortingen en begrippen

Afkortingenlijst

Afkorting	Betekenis
Applicatie en gegevensverwerking	
CSV	Comma-Separated Values: tabelformaat voor de opgeslagen timingdata.
DOM	Document Object Model: de programmeerbare structuur van een webpagina.
HMI	Human-Machine Interaction: interactie tussen de gebruiker en de applicatie.
IPC	Inter-Process Communication: communicatie tussen de Electron-processen.
UI	User Interface: de visuele gebruikersinterface van de applicatie.
Motorsport en live timing	
MSTC	MotorSport Training Center.
PIC	Position in Class: positie van een wagen binnen zijn klasse.
BIC	Best in Class: de best presterende wagen binnen dezelfde klasse.
XIC	Selected Car in Class: een gekozen klassewagen waarmee het team wordt vergeleken.
ETA	Estimated Time of Arrival: geschatte tijd tot een volgend timingpunt.
RIS	Races Information Services: aanbieder van live timingdata.
SC	Safety Car: neutralisatie waarbij het deelnemersveld achter de safety car rijdt.
FCY	Full Course Yellow: neutralisatie waarbij alle wagens hun snelheid beperken.

Motorsportbegrippen

Term	Betekenis
Klasse	Groep wagens met vergelijkbare technische eigenschappen die binnen dezelfde race een afzonderlijk klassement vormen.
Sector	Een deel van het circuit waarvoor afzonderlijk een tussentijd wordt gemeten.
Stint	De aaneengesloten periode waarin een piloot met de wagen rijdt tussen twee pitstops of pilotenwissels.
Inlap	De ronde waarin de wagen aan het einde van de ronde de pitstraat binnenrijdt.
Outlap	De eerste ronde nadat de wagen de pitstraat heeft verlaten.
Pitstop window	Tijdperiode waarin een verplichte pitstop volgens het wedstrijdreglement geldig mag worden uitgevoerd.
Pit loss	Het geschatte tijdverlies van een pitstop ten opzichte van een wagen die op het circuit blijft rijden.
Gap	Het tijds- of rondeverschil tussen twee wagens.
Delta	Het berekende tijdsverschil tussen twee rondes, sectoren, wagens of piloten.
Reference time	Ingestelde minimale ronde- of sectortijd waarmee actuele en voorspelde tijden worden vergeleken.
Ideal lap	Theoretische rondetijd die ontstaat door de beste geregistreerde sectortijden van dezelfde wagen op te tellen.
Track limits	De toegestane grenzen van het circuit. Een overtreding kan leiden tot het ongeldig verklaren van een rondetijd.
Race Control	Wedstrijdleiding die onder andere vlagstatussen, neutralisaties, straffen en baanveiligheid beheert.
Green flag	Normale racesituatie waarin het circuit vrij is en op wedstrijdsnelheid mag worden gereden.
Red flag	Onderbreking van de sessie omdat verder rijden tijdelijk niet veilig of niet mogelijk is.

Bijlage C

Stageplan

Stageplan: Jeff Mathieu <> MotorSport Training Center

Student	Jeff Mathieu
Opleiding	Master Computer Science
Stageperiode	22 juni - 31 juli (6 weken)
Dagelijkse begeleider	Jan Wouters, [FUNCTIE], info@motorsport-trainingcenter.be, +32 475 39 41 16
Fysische locatie	[Locatie]

1. Probleemstelling

Tijdens een endurancewedstrijd moet een race-engineer snel en betrouwbaar beslissingen nemen op basis van live timingdata. De bestaande live timingomgeving van Circuit Zolder toont veel ruwe informatie, zoals rondetijden, sectortijden, posities en andere sessie-informatie. Deze informatie is echter niet specifiek afgestemd op de concrete beslissingen te maken die een team tijdens de race moet nemen. Voor een team is het vooral belangrijk om de prestaties van de eigen wagen en de relevante klasse overzichtelijk te volgen. Een belangrijke regel die teams worden opgelegd is de referentietijdregel. Deze regel is er zodat teams niet sneller kunnen rijden dan de andere auto's in hun klasse. Het is dus belangrijk dat het team een waarschuwing krijgt wanneer deze referentietijdregel in gevaar komt. Door de verwachte rondetijden in te kunnen schatten kunnen er strategische beslissingen gemaakt worden.

Een belangrijk probleem wat waarschijnlijk gaat voorkomen is dat de site met live timingdata tijdelijk kan wegvallen. Een praktisch dashboard mag daarom niet afhankelijk zijn van één browservenster. De applicatie moet dus ontvangen data lokaal bewaren, ook kunnen werken met replay- of simulatiegegevens wanneer er geen race bezig is en na een herstart of onderbreking opnieuw bruikbaar zijn zonder historiek te verliezen.

Daarnaast willen we onderzoeken of historische en live rondetijddata gebruikt kunnen worden om rondetijden van piloten te voorspellen over korte tijdsintervallen. Deze voorspellingen kunnen helpen om tijdsverlies of tijdswinst over bijvoorbeeld 15 minuten, 30 minuten en 1 uur te schatten en zo de potstrategie beter te ondersteunen.

2. Concrete doelstelling

Het doel van de stage is het ontwerpen en implementeren van een lokaal draaiende softwaretool die timingdata verwerkt, analyseert en visualiseert. Het eindresultaat is een werkend prototype dat bruikbaar is in simulatie/replaymodus en live de race kan volgen.

Concreet verwachten we volgende resultaten:

- Een dashboard dat voor de auto rondetijden, sectortijden, gemiddelden, snelste rondes en trends toont
- Een module die referentietijden per sessie configureerbaar maakt en waarschuwingen geeft bij kritieke rondetijden of mogelijke overtredingen
- Een lokale databank waarin timingevents, rondes, sectortijden, correcties en sessiegegevens persistent worden opgeslagen
- Een replay- en simulatiemodus waarmee het systeem getest kan worden zonder actieve race
- Een eerste voorspellend model dat op basis van beschikbare data rondetijden en tijdsverlies over 15, 30 en 60 minuten inschat
- Een duidelijke gebruikersinterface waarmee een race-engineer snel de status van de wagen kan zien en hiermee strategie beslissingen kan maken.

3. Activiteiten en opdrachten

Ik zal binnen het team deelnemen aan de volgende activiteiten en opdrachten: live timingdata verwerken, dashboard ontwikkelen, rondetijden analyseren, AI/ML-voorspellingen maken, tijdsverlies inschatten, pitstrategie ondersteunen, resultaten testen en valideren met de dagelijkse begeleider.

Ik kan het systeem testen door de replay-functionaliteit te implementeren en gebruiken. Als het prototype ver genoeg gevorderd is om het tijdens een raceweekend te testen, kan dit vanuit thuis of tijdens de race op het circuit zodat ik tijdens de race kan laten zien hoe het prototype werkt.

4. Technische aanpak

De applicatie wordt opgevat als een local-first dashboard. De gebruikersinterface staat los van de opslag en verwerking van de timingdata. De timingcollector haalt gegevens op uit de bron, normaliseert deze naar een intern formaat en bewaart ze in een lokale database. De analyselaag berekent vervolgens gemiddelden, marges ten opzichte van referentietijden, risico-indicatoren en voorspellingen. De dashboardlaag toont deze resultaten aan de gebruiker.

De implementatie zal bij voorkeur bestaan uit een desktop- of lokale webapplicatie met een moderne frontend, een backendlaag voor dataverwerking en een SQLite-database voor lokale opslag. De software moet ontwikkeld kunnen worden op macOS en later kunnen draaien op een Windows-laptop. Daarom worden platformafhankelijke paden en instellingen vermeden. Indien mogelijk wordt ook een eenvoudige manier voorzien om de applicatie te verpakken of te starten op Windows.

Voor de AI/ML-component wordt gestart met een eenvoudige en uitlegbare baseline, bijvoorbeeld een rolling-average- of regressiemodel op basis van recente rondetijden, sectortijden, piloot, stuntsfase, pitstatus en eventuele vlag-of neutralisatieperiodes. Daarna kan onderzocht worden of een uitgebreider model betere voorspellingen oplevert. De nauwkeurigheid wordt geëvalueerd door voorspelde rondetijden en tijdsverlies te vergelijken met historische of gereplayde sessiedata.

5. Werkplan en timing

Deze stage duurt 6 weken. Onderstaande planning is een inschatting van de activiteiten die ik die week ga aanvangen. In overleg met de begeleider kan deze planning worden aangepast naarmate prioriteiten veranderen doorheen het project.

	Taak	Omschrijving
Week 1	Analyse en specificatie	Exacte vereisten overleggen, beschikbare timing data onderzoeken, datamodellen onderzoeken, referentietijd-regel begrijpen.
Week 2	Dataverwerking en opslag	Lokale database ontwerpen, import/replay van historische sessiedata voorzien om testen te kunnen uitvoeren de volgende weken.
Week 3	Dashboard en basisanalyse	Hoofdschermen bouwen voor overzicht, rondetijden, sectortijden, gemiddelden, snelste rondes en waarschuwingen.
Week 4	Robuustheid en simulatie	Replaymodus uitbreiden, onderbrekingen en correcties simuleren, persistentie en herstel na herstart testen.
Week 5	AI/ML-voorspelling	Voorspellingsmodel ontwikkelen voor rondetijden en tijdsverlies, resultaten evalueren en visualiseren.
Week 6	Validatie en afwerking	Dashboard testen met begeleider, feedback verwerken, documentatie schrijven, beperkingen en verdere uitbreidingen beschrijven, eindprototype opleveren.

6. Verwachte deliverables

- Werkend prototype van het timingdashboard met lokale opslag
- Import- en replaymogelijkheid voor historische of gesimuleerde timingdata
- Analysefuncties voor rondetijden, sectortijden, gemiddelden, referentietijdmarges en waarschuwingen
- AI/ML-module voor voorspellingen van rondetijden en tijdsverlies
- Korte technische documentatie om "klant" zelfstandig deze tool te laten gebruiken

7. Stageplaats en begeleiding

Ik al slechts op bepaalde momenten ter plaatse aanwezig zijn, bijvoorbeeld voor overleg, feedbackmomenten of praktische afstemming met de begeleider. Dit aangezien de begeleider zelf vaak op verplaatsing werkt. Het grootste deel van de technische uitwerking zal zelfstandig van thuis uit gebeuren.

De begeleider levert vooral domeinkennis, requirements (meeste vooraf afgesproken, er kunnen natuurlijk altijd requirements veranderen/bijkomen), feedback vanuit een race-engineering perspectief. De communicatie zal voornamelijk via e-mail of telefoon verlopen. Ik zal regelmatig vragen stellen, mijn voortgang bespreken en feedback verwerken om te controleren of het dashboard aansluit bij de praktische doeleinden van het race team.

Bijlage D

Competenties die ik verwachtte te verbeteren

Competenties die ik verwacht te verbeteren tijdens mijn stage

1. Zelfstandig werken

Ik zal een groot deel van de analyse, het ontwerp en de implementatie zelfstandig uitvoeren.

2. Samenwerken en afstemmen

Ik leer mijn werk regelmatig afstemmen met de begeleider en feedback verwerken in het project.

3. Communicatie met een niet-IT klant

Ik leer technische keuzes duidelijk uitleggen aan iemand met vooral domeinkennis uit de motorsport.

4. Requirements analyseren

Ik zal noden uit de praktijk vertalen naar concrete functies voor het timingdashboard.

5. Data-analyse en AI/ML

Ik leer timingdata verwerken en gebruiken om rondetijden en mogelijk tijdsverlies te voorspellen.

6. Projectmatig werken

Ik leer een technisch project plannen, testen, bijsturen en opleveren binnen een vaste stageperiode.